# Mapeamento Arquitetural para Git e GitHub

*Planejamento da estrutura do repositório do TaskTracker — Preparação para a Fase 2*

**Nome:** Eduardo Henrique Moraes Santos
**Curso:** Ciência de Dados e Machine Learning
**Campus:** Asa Norte
**Email:** eduardo.henriques@sempreceub.com
**Disciplina:** Bootcamp II

# Desenho da Árvore de Repositório

Estrutura de pastas e arquivos do TaskTracker que será versionada no GitHub




***Observação Importante:*** *O bloco tracejado não é uma pasta, ele reúne os arquivos que ficam soltos na raiz do repositório, no mesmo nível de docs/ e src/.*

# Arquivos da Raiz e Documentação

Finalidade de cada item que fica fora da pasta de código-fonte

### `README.md`

Apresenta o projeto na página inicial do GitHub: o que é o TaskTracker, suas funcionalidades e como executá-lo.

### `.gitignore`

Informa ao Git quais arquivos não devem ir para o repositório, como a pasta __pycache__ gerada pelo Python.

### `requirements.txt`

Lista as dependências do projeto. Nesta fase o TaskTracker usa apenas a biblioteca padrão do Python.

### `Dockerfile`

Receita de empacotamento do programa em container, prevista para a Fase 3 do projeto.

### `docs/`

Reúne a documentação do projeto, mantendo-a separada do código-fonte.

### `docs/planejamento_logico.pdf`

Documento de planejamento lógico da Fase 1, guardado no repositório como registro do projeto.

### `docs/mapeamento_arquitetural.pdf`

Esta apresentação, com o diagrama de hierarquia planejado para o repositório.

### `src/`

Reúne o código-fonte (source) do projeto, seguindo a convenção usada em projetos Python.

# Divisão do Código-Fonte em src/

Cada módulo corresponde a uma parte da receita lógica descrita no documento

### `main.py` Ponto de entrada do programa

- Cria a lista vazia que guarda as tarefas
- Controla o laço de repetição do menu
- Encerra o programa na opção 5 (Regras 8 e 9)

### `menu.py` Interface com o usuário

- Exibe a mensagem de boas-vindas e as 5 opções
- Lê a opção digitada e verifica se está entre 1 e 5
- Avisa quando a opção é inválida

### `tarefas.py` As quatro funcionalidades

- cadastrar_tarefa() e listar_tarefas()
- concluir_tarefa() e exibir_resumo()
- Ordena por prioridade e evita conclusão repetida (Regras 3, 6 e 7)

### `validacoes.py` Regras de validação das entradas

- Título não pode ficar em branco (Regra 1)
- Prioridade só pode ser Alta, Média ou Baixa (Regra 2)
- Data no formato DD/MM/AAAA e número de tarefa válido (Regras 4 e 5)

# Plano de Aprendizado para Git e GitHub

O que espero aprender sobre controle de versão na Fase 2 do projeto

01

## Commits

Registrar cada alteração de forma organizada, com mensagens claras, para que o histórico conte a evolução do TaskTracker.

02

## Branches

Desenvolver e testar cada módulo separadamente, sem comprometer a versão do programa que já funciona.

03

## Conflitos e histórico

Resolver conflitos de código e navegar pelo histórico para recuperar versões anteriores caso algo quebre.

04

## Publicação no GitHub

Publicar o repositório de forma profissional, com README bem escrito e estrutura de pastas coerente.